

Listening to Photon Noise

A low-voltage random-number generator and music machine from a light sensor

Youssef Nassar

September 25, 2026

Abstract

Light arrives in photons, so even a steady beam delivers a slightly different number of them in every instant. We point a phone flashlight through a small paper-covered hole in a box onto a \$7, 3.3 V light sensor (ams TSL2591) read by a Raspberry Pi, record one 5.0-minute run, and use the last digit of each reading as a source of random bits. A von Neumann extractor turns 5,216 raw bits into 1,320 clean bits. The debiased stream passes all four tests at $\alpha = 0.01$: no detectable bias or correlation in 1,320 bits. The same bits drive live music in Sonic Pi. No high-voltage parts are used.

<https://photos.icloud.com/shared/album/0b4N0oxaatJdm5jsKze8INpxQ>

1 Idea

The original design counted single photons with a photomultiplier tube, which needs 500–1200 V and a lab safety review. This version uses a low-voltage integrating light sensor instead. It cannot see single photons; it sums millions of them per 100 ms reading. What it can see is the *noise* on that sum, and the last digit of a noisy number is hard to predict. That is enough to build a random-number generator and to turn it into music. We do not claim a rigorous quantum random-number generator: the noise is a mix of photon shot noise and electronic noise, and this build does not separate the two precisely.

2 Background

Shot noise. If photoelectrons arrive at random at a steady average rate, the number N collected in one reading follows a Poisson distribution, with standard deviation $\sqrt{\langle N \rangle}$. This fluctuation comes from the particle nature of light, not from the electronics.

Read noise. The sensor’s own amplifier and analog-to-digital converter add a few counts of noise that are present even in the dark. Both kinds of noise are different on each of the TSL2591’s two photodiodes, whereas flicker of the light source moves both channels together. The correlation between the two channels’ noise therefore tells us whether the light source itself is the dominant noise.

From noise to bits. When the noise is a few counts wide, the least-significant bit (LSB) of each reading is close to a fair coin. A von Neumann extractor [1] removes remaining bias: bits are read in pairs, $01 \rightarrow 0$, $10 \rightarrow 1$, and 00 , 11 are discarded. It keeps about a quarter of the bits. We check the result with four quick tests in the style of NIST SP 800-22 [2].

3 System

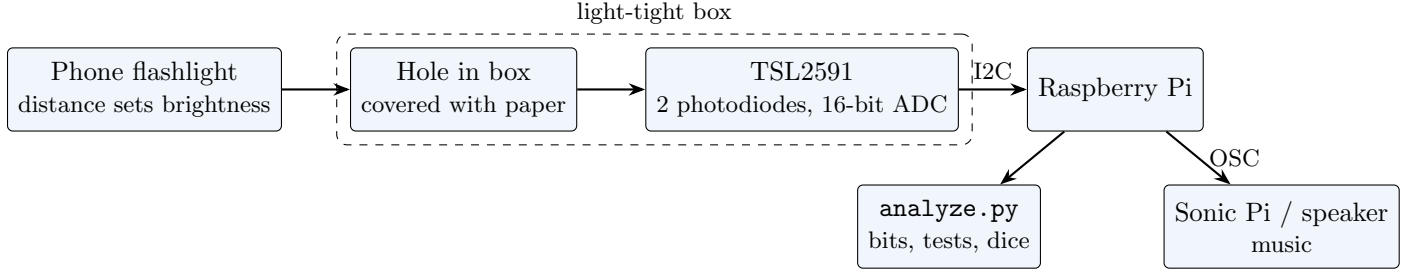


Figure 1: Signal chain. Everything runs from the Pi’s 3.3 V rail.

Part	Role
Adafruit TSL2591 breakout	two photodiodes (visible+IR, IR), 16-bit ADCs, I ² C at 0x29
STEMMA QT cable	3.3 V, GND, SDA (GPIO2), SCL (GPIO3) to the Pi header
Opaque box, black foam tape	light-tight enclosure
Small hole in the box, printer paper over it	light entry and diffusion
Phone flashlight	light source; its distance sets the brightness
Raspberry Pi, USB speaker	acquisition, analysis, music

Table 1: Hardware used.

4 Method

The sensor runs at gain MAX (the highest) and 100 ms integration time. Each reading restarts the converter and waits for a fresh result, so no reading is ever repeated. With paper taped over the hole, the phone is moved until channel 0 reads between 3,000 and 25,000 counts (well above the dark level, well below saturation), rested, and one run of 2,608 readings is recorded over 5.0 min.

Noise is measured from sample-to-sample changes, $\sigma = \text{std}(x_{i+1} - x_i) / \sqrt{2}$, which ignores slow drift of the phone’s light. The LSBs of both channels give 5,216 raw bits, and von Neumann extraction keeps 1,320 of them (25 %), saved as 165 bytes. For dice, clean bits are read three at a time (0–7) and values 6 and 7 are discarded, so every face stays equally likely.

Music. Every four clean bits make a note: three bits choose one of eight notes of A minor pentatonic, one bit chooses a short or long decay. Notes go to Sonic Pi over OSC; the light level sets the brightness of a background pad, so covering the hole darkens the sound. The same bits also drive a Karplus–Strong plucked-string synthesizer and golden-ratio FM bells rendered to the USB speaker. Demo: *[add a link to your demo video here]*.

5 Results

Channel 0 averaged 10,826 counts with noise $\sigma = 1.56$ counts; channel 1 (IR) averaged 2,306 counts with $\sigma = 0.74$ counts. The noise on the two photodiodes is nearly uncorrelated ($r = 0.10$), so it is mostly independent per-diode noise (photon shot noise plus read noise) rather than flicker of the phone light, which would move both channels together.

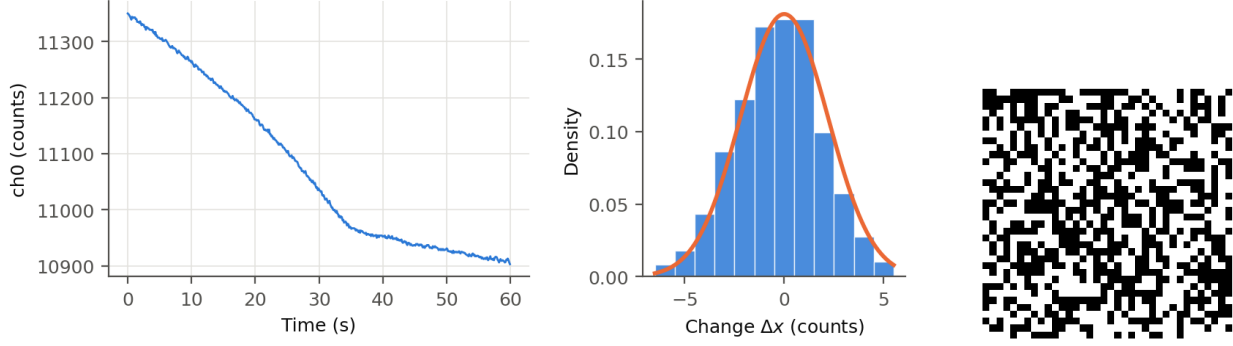


Figure 2: Left: first 60s of the run and the distribution of its sample-to-sample changes, with a Gaussian of equal width. Right: the first clean bits as a black/white image; stripes or blocks would reveal correlation.

Test	Raw LSBs (5,216 bits)	Debiased (1,320 bits)
Frequency (monobit)	0.740 (pass)	0.660 (pass)
Runs	0.979 (pass)	0.036 (pass)
Serial (2-bit, χ^2)	0.873 (pass)	0.360 (pass)
Lag-1 autocorrelation	0.967 (pass)	0.039 (pass)

Table 2: Randomness tests; a test passes if $p \geq 0.01$.

The debiased stream passes all four tests at $\alpha = 0.01$: no detectable bias or correlation in 1,320 bits. The first dice rolls generated from the sensor noise: **4 2 2 1 3 2 2 5 2 5 5 3 6 3 3 1 6 6 3 1.**

6 Discussion

What this shows. A cheap light sensor’s noise is wide enough that its last digit behaves like a coin flip, and after debiasing the bits show no detectable bias or pattern at this sample size. Photon shot noise is part of that noise, which is why covering the hole changes it.

Limitations. (1) The sensor integrates millions of photons, so single photons are never seen. (2) One run at one brightness cannot separate shot noise from read noise; measuring the noise at several distances and checking that the noise variance grows in proportion to the signal would. (3) Four quick tests on about a thousand bits catch obvious flaws only; a real random-number generator needs the full NIST suite on millions of bits. (4) The phone flashlight may flicker; the two-channel correlation above flags this.

Next steps. Record at several phone distances to measure the shot-noise fraction; run longer to collect enough bits for the full NIST suite; replace the sensor with a single-photon detector once a high-voltage safety review is possible.

7 Conclusion

A \$7 light sensor, a box with a paper-covered hole, a phone flashlight and a Raspberry Pi make a working low-voltage random number generator whose output you can hear.

References

- [1] J. von Neumann, “Various techniques used in connection with random digits,” *NBS Applied Mathematics Series* 12, pp. 36–38, 1951.
- [2] A. Rukhin *et al.*, *A Statistical Test Suite for Random and Pseudorandom Number Generators*, NIST SP 800-22 Rev. 1a, 2010.
- [3] ams AG, *TSL2591 Light-to-Digital Converter*, datasheet.
- [4] S. Aaron, *Sonic Pi*, <https://sonic-pi.net>.

A Code

Listing 1: sensor.py

```
1  """Minimal TSL2591 driver over I2C (smbus), plus a simulator for dry runs.
2
3  Why not the Adafruit library? Its free-running mode returns the *last finished*
4  ADC value, so polling faster than the integration time silently repeats samples,
5  which ruins noise statistics. Here every read() restarts the ADC and waits for a
6  fresh integration (one-shot mode), so every sample is independent.
7
8  Register map (TSL2591 datasheet): command byte = 0xA0 | register.
9      0x00 ENABLE bit0 PON (oscillator on), bit1 AEN (ADC on)
10     0x01 CONTROL bits5:4 gain, bits2:0 integration time (0=100 ms ... 5=600 ms)
11     0x12 ID reads 0x50
12     0x13 STATUS bit0 AVALID: an integration finished since AEN was set
13     0x14..0x17 CODATAL, CODATAH (full spectrum), C1DATAL, C1DATAH (IR)
14 """
15 import time
16
17 import numpy as np
18
19 ADDR = 0x29
20 CMD = 0xA0
21 REG_ENABLE, REG_CONTROL, REG_ID, REG_STATUS, REG_CO = 0x00, 0x01, 0x12, 0x13, 0x14
22 PON, AEN = 0x01, 0x02
23 GAIN_BITS = {"LOW": 0x00, "MED": 0x10, "HIGH": 0x20, "MAX": 0x30}
24 GAIN_X = {"LOW": 1, "MED": 25, "HIGH": 428, "MAX": 9876}
25 ATIMES_MS = [100, 200, 300, 400, 500, 600]
26
27
28 def saturation(ptime_ms):
29     """Largest valid count; the 100 ms cycle saturates below 16 bits."""
30     return 36863 if ptime_ms == 100 else 65535
31
32
33 class TSL2591:
34     simulated = False
35
36     def __init__(self, gain="MAX", atime_ms=100, bus=1):
37         try:
38             from smbus2 import SMBus
39         except ImportError: # python3-smbus has the same calls
```

```

40     from smbus import SMBus
41     self.bus = SMBus(bus)
42     dev_id = self.bus.read_byte_data(ADDR, CMD | REG_ID)
43     if dev_id != 0x50:
44         raise RuntimeError(f"chip at 0x29 reports ID 0x{dev_id:02X}, expected 0x50 (TSL2591)")
45     self.gain, self.ptime_ms = gain, ptime_ms
46     self.bus.write_byte_data(ADDR, CMD | REG_CONTROL,
47                             GAIN_BITS[gain] | ATIMES_MS.index(ptime_ms))
48     self.bus.write_byte_data(ADDR, CMD | REG_ENABLE, PON)
49     time.sleep(0.005)
50
51     def read(self):
52         """Run one fresh integration. Returns (t_seconds, ch0_full, ch1_ir)."""
53         self.bus.write_byte_data(ADDR, CMD | REG_ENABLE, PON) # ADC off, clears AVALID
54         self.bus.write_byte_data(ADDR, CMD | REG_ENABLE, PON | AEN) # start a new cycle
55         t0 = time.monotonic()
56         time.sleep(1.12 * self.ptime_ms / 1000) # full cycle + margin, even if AVALID lags
57         while not self.bus.read_byte_data(ADDR, CMD | REG_STATUS) & 0x01:
58             if time.monotonic() - t0 > 2 * self.ptime_ms / 1000 + 0.1:
59                 raise TimeoutError("TSL2591 never finished an integration")
60             time.sleep(0.002)
61         d = self.bus.read_i2c_block_data(ADDR, CMD | REG_CO, 4) # one transaction latches all 4
62         return time.monotonic(), d[0] | d[1] << 8, d[2] | d[3] << 8
63
64     def close(self):
65         self.bus.write_byte_data(ADDR, CMD | REG_ENABLE, 0x00)
66         self.bus.close()
67
68
69 class SimulatedTSL2591:
70     """Fake sensor with Poisson photon statistics, for rehearsing without hardware.
71     Its numbers are invented: never put them in a report (analyze.py flags them)."""
72     simulated = True
73
74     def __init__(self, gain="MAX", ptime_ms=100, realtime=False, seed=None):
75         self.gain, self.ptime_ms, self.realtime = gain, ptime_ms, realtime
76         self.rng = np.random.default_rng(seed)
77         self.k = 3000.0 * 9876 / GAIN_X[gain] # electrons per count (made up)
78         self.level, self.drift, self.t = 0.0, 0.0, 0.0
79
80     def set_level(self, counts):
81         self.level, self.drift = float(counts), 0.0
82
83     def read(self):
84         dt = self.ptime_ms / 1000
85         if self.realtime:
86             time.sleep(dt)
87         self.t += dt * 1.05
88         self.drift += self.rng.normal(0, 1e-4) # slow source wander
89         out = []
90         for share, dark in ((1.0, 3.0), (0.35, 1.0)):
91             mean_e = max(self.level * share * (1 + self.drift), 0) * self.k
92             counts = self.rng.poisson(mean_e) / self.k + dark + self.rng.normal(0, 1.2)
93             out.append(int(np.clip(round(counts), 0, saturation(self.ptime_ms))))
94         return self.t, out[0], out[1]
95
96     def close(self):
97         pass
98
99
100 def open_sensor(simulate=False, gain="MAX", ptime_ms=100, realtime=False):
101     if simulate:
102         return SimulatedTSL2591(gain, ptime_ms, realtime=realtime)
103     return TSL2591(gain, ptime_ms)

```

Listing 2: collect.py

```

1  """Data collection for the photon-noise sonifier.
2
3  python3 collect.py check live readout (wiring, aiming, light-leak test)
4  python3 collect.py run the one recording: aim the phone, press Enter, wait
5
6  Add --simulate to rehearse without hardware.
7  """
8  import argparse
9  import csv
10 import os
11 import select
12 import sys
13
14 from sensor import open_sensor, saturation
15
16 HERE = os.path.dirname(os.path.abspath(__file__))
17 DATA = os.path.join(HERE, "data")
18 TARGET = (3000, 25000) # good ch0 range: plenty of noise, far from saturation
19
20
21 def enter_pressed():
22     return bool(select.select([sys.stdin], [], [], 0)[0])
23
24
25 def cmd_check(sensor, args):
26     print(f"TSL2591 OK gain={sensor.gain} integration={sensor.ptime_ms} ms "
27           f"saturation={saturation(sensor.ptime_ms)} (Ctrl+C to stop)")
28     try:
29         while True:
30             _, c0, c1 = sensor.read()
31             bar = "#" * min(60, int(60 * c0 / saturation(sensor.ptime_ms)))
32             print(f"ch0 {c0:6d} ch1 {c1:6d} |{bar}")
33     except KeyboardInterrupt:
34         print()
35
36
37 def aim(sensor):
38     """Live readout until Enter, with a hint to move the phone closer or farther."""
39     print(f"\nPoint the phone flashlight at the hole. Move it closer/farther until ch0 is "
40           f"between {TARGET[0]} and {TARGET[1]}, rest the phone, then press Enter.")
41     sat = saturation(sensor.ptime_ms)
42     while True:
43         _, c0, c1 = sensor.read()
44         hint = ("SATURATED - move farther" if c0 >= 0.95 * sat else "move closer" if c0 < TARGET[0]
45                else "move farther" if c0 > TARGET[1] else "good - press Enter")
46         sys.stdout.write(f"\r ch0 {c0:6d} ch1 {c1:6d} {hint:<26}")
47         sys.stdout.flush()
48         if enter_pressed():
49             sys.stdin.readline()
50             print()
51             return
52
53
54 def cmd_run(sensor, args):
55     if sensor.simulated:
56         sensor.set_level(args.sim_level)
57     else:
58         aim(sensor)
59     n = int(args.minutes * 60 / (sensor.ptime_ms / 1000 * 1.15))
60     print(f"Recording {args.minutes:g} min ({n} samples). Don't touch anything.")
61     rows = []
62     for i in range(n):
63         t, c0, c1 = sensor.read()
64         rows.append((t, c0, c1))
65         if i % 50 == 0:
66             sys.stdout.write(f"\r {i}/{n} ch0 {c0:6d}")
67             sys.stdout.flush()
68     print(f"\r {n}/{n} done ")

```

```

69     os.makedirs(DATA, exist_ok=True)
70     path = os.path.join(DATA, "run.csv")
71     t0 = rows[0][0]
72     with open(path, "w", newline="") as f:
73         f.write(f"# gain={sensor.gain} atime_ms={sensor.atime_ms} simulated={int(sensor.simulated)}\n")
74         w = csv.writer(f)
75         w.writerow(["t_s", "ch0", "ch1"])
76         w.writerows((f"{t - t0:.4f}", c0, c1) for t, c0, c1 in rows)
77     print(f"Saved data/run.csv. Next: python3 analyze.py")
78
79
80 def main():
81     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
82     p.add_argument("command", choices=["check", "run"])
83     p.add_argument("--minutes", type=float, default=5, help="length of the recording")
84     p.add_argument("--gain", default="MAX", choices=["LOW", "MED", "HIGH", "MAX"])
85     p.add_argument("--simulate", action="store_true", help="fake sensor, for rehearsal only")
86     p.add_argument("--sim-level", type=float, default=12000, help=argparse.SUPPRESS)
87     args = p.parse_args()
88     sensor = open_sensor(args.simulate, args.gain, 100, realtime=args.command == "check")
89     try:
90         {"check": cmd_check, "run": cmd_run}[args.command](sensor, args)
91     finally:
92         sensor.close()
93
94
95 if __name__ == "__main__":
96     main()

```

Listing 3: analyze.py

```

1  """Analyze data/run.csv: noise statistics, random-number generation, figures, report.
2
3  python3 analyze.py
4
5  Random numbers: the least-significant bit (LSB) of each channel of each sample is
6  a raw random bit (the noise is several counts wide, so the last digit is
7  unpredictable). A von Neumann extractor removes bias: bits are read in pairs,
8  01 -> 0, 10 -> 1, 00 and 11 are discarded. The clean bits are tested, saved to
9  data/random_bits.txt and data/random_bytes.bin, and turned into dice rolls.
10
11 Writes report/results.tex, report/tests_table.tex, report/figures/*.png, copies
12 the code into report/code/ and zips the report folder as report_overleaf.zip.
13 """
14 import csv
15 import math
16 import os
17 import shutil
18
19 import numpy as np
20 import matplotlib
21
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt # noqa: E402
24
25 from sensor import saturation # noqa: E402
26
27 HERE = os.path.dirname(os.path.abspath(__file__))
28 DATA = os.path.join(HERE, "data")
29 REPORT = os.path.join(HERE, "report")
30 FIG = os.path.join(REPORT, "figures")
31 BLUE, ORANGE, INK, INK2, GRID = "#2a78d6", "#eb6834", "#0b0b0b", "#52514e", "#e4e3df"
32 ALPHA = 0.01
33
34 plt.rcParams.update({
35     "font.size": 9, "axes.edgecolor": INK2, "axes.labelcolor": INK, "xtick.color": INK2,
36     "ytick.color": INK2, "axes.spines.top": False, "axes.spines.right": False,
37     "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.6,

```

```

38     "savefig.dpi": 200, "savefig.bbox": "tight",
39 })
40
41
42 def fmt(x, nd=2):
43     return "n/a" if x is None or not np.isfinite(x) else f"{x:.{nd}f}".replace(",", "{,}")
44
45
46 def von_neumann(bits):
47     pairs = bits[: len(bits) // 2 * 2].reshape(-1, 2)
48     return pairs[pairs[:, 0] != pairs[:, 1], 0]
49
50
51 def randomness_tests(bits):
52     """Four quick tests in the style of NIST SP 800-22; each returns a p-value."""
53     n = len(bits)
54     if n < 100:
55         return {k: float("nan") for k in ("monobit", "runs", "serial", "lag1")}
56     s = 2 * bits.astype(float) - 1
57     out = {"monobit": math.erfc(abs(s.sum()) / math.sqrt(2 * n))}
58     pi = bits.mean()
59     if abs(pi - 0.5) >= 2 / math.sqrt(n):
60         out["runs"] = 0.0
61     else:
62         v = 1 + int(np.sum(bits[1:] != bits[:-1]))
63         out["runs"] = math.erfc(abs(v - 2 * n * pi * (1 - pi)) / (2 * math.sqrt(2 * n) * pi * (1 - pi)))
64     pairs = bits[: n // 2 * 2].reshape(-1, 2)
65     counts = np.bincount(pairs[:, 0] * 2 + pairs[:, 1], minlength=4)
66     m = len(pairs) / 4
67     x = float(np.sum((counts - m) ** 2 / m))
68     out["serial"] = math.erfc(math.sqrt(x / 2)) + math.sqrt(2 * x / math.pi) * math.exp(-x / 2)
69     r = float(np.mean(s[1:] * s[:-1]))
70     out["lag1"] = math.erfc(abs(r) * math.sqrt(n - 1) / math.sqrt(2))
71     return out
72
73
74 def tests_table(raw_t, vn_t, n_raw, n_vn):
75     names = {"monobit": "Frequency (monobit)", "runs": "Runs", "serial": r"Serial (2-bit,  $\chi^2$ )",
76             "lag1": "Lag-1 autocorrelation"}
77
78     def cell(v):
79         if not np.isfinite(v):
80             return "n/a"
81         return f"{v:.3f} (" + ("pass" if v >= ALPHA else r"\textbf{fail}") + ")"
82
83     lines = [r"\begin{tabular}{lcc}", r"\toprule",
84             rf"Test & Raw LSBs ({fmt(n_raw, 0)} bits) & Debiased ({fmt(n_vn, 0)} bits) \\", r"\midrule"]
85     lines += [f"{names[k]} & {cell(raw_t[k])} & {cell(vn_t[k])} \\\\" for k in names]
86     lines += [r"\bottomrule", r"\end{tabular}"]
87     return "\n".join(lines) + "\n"
88
89
90 def plots(t, c0, d0, vn):
91     fig, (a1, a2) = plt.subplots(1, 2, figsize=(6.4, 2.6), gridspec_kw={"width_ratios": [1.6, 1]})
92     k = t <= 60
93     a1.plot(t[k], c0[k], color=BLUE, lw=1)
94     a1.set_xlabel("Time (s)")
95     a1.set_ylabel("ch0 (counts)")
96     lo, hi = np.percentile(d0, [0.5, 99.5])
97     bins = np.arange(math.floor(lo) - 0.5, math.ceil(hi) + 1.5, max(1, round((hi - lo) / 40)))
98     a2.hist(d0, bins=bins, density=True, color=BLUE, alpha=0.85, edgecolor="white", linewidth=0.5)
99     sd = d0.std()
100    xs = np.linspace(bins[0], bins[-1], 300)
101    a2.plot(xs, np.exp(-xs ** 2 / (2 * sd ** 2)) / (sd * math.sqrt(2 * math.pi))), color=ORANGE, lw=2)
102    a2.set_xlabel(r"Change  $\Delta x$  (counts)")
103    a2.set_ylabel("Density")
104    a2.grid(False)
105    fig.tight_layout()

```

```

106 fig.savefig(os.path.join(FIG, "noise.png"))
107 plt.close(fig)
108
109 side = int(math.isqrt(min(len(vn), 4096)))
110 if side >= 8:
111     fig, ax = plt.subplots(figsize=(2.4, 2.4))
112     ax.imshow(vn[: side * side].reshape(side, side), cmap="gray_r", interpolation="nearest")
113     ax.set_axis_off()
114     fig.savefig(os.path.join(FIG, "bits.png"))
115     plt.close(fig)
116
117
118 def main():
119     path = os.path.join(DATA, "run.csv")
120     if not os.path.exists(path):
121         raise SystemExit("No data/run.csv yet: run python3 collect.py run first.")
122     with open(path) as f:
123         meta = dict(kv.split("=", 1) for kv in f.readline().lstrip("#").split())
124         rows = list(csv.DictReader(f))
125     t = np.array([float(r["t_s"]) for r in rows])
126     c0 = np.array([int(r["ch0"]) for r in rows])
127     c1 = np.array([int(r["ch1"]) for r in rows])
128     simulated = meta.get("simulated") == "1"
129     sat = bool(c0.max() >= 0.95 * saturation(int(meta["atime_ms"])))
130
131     # noise: sample-to-sample changes (differencing removes slow drift of the phone light)
132     d0, d1 = np.diff(c0).astype(float), np.diff(c1).astype(float)
133     sigma0, sigma1 = d0.std() / math.sqrt(2), d1.std() / math.sqrt(2)
134     corr = float(np.corrcoef(d0, d1)[0, 1]) if d0.std() > 0 and d1.std() > 0 else float("nan")
135
136     # random numbers
137     raw = np.empty(2 * len(c0), dtype=np.uint8)
138     raw[0::2], raw[1::2] = c0 & 1, c1 & 1
139     vn = von_neumann(raw)
140     raw_t, vn_t = randomness_tests(raw), randomness_tests(vn)
141     fails = [k for k, v in vn_t.items() if not (v >= ALPHA)]
142     with open(os.path.join(DATA, "random_bits.txt"), "w") as f:
143         f.write("".join(map(str, vn)) + "\n")
144     nbytes = len(vn) // 8
145     np.packbits(vn[: nbytes * 8]).tofile(os.path.join(DATA, "random_bytes.bin"))
146     # dice: 3 bits -> 0..7, keep 0..5 (rejection sampling keeps every face equally likely)
147     tri = vn[: len(vn) // 3 * 3].reshape(-1, 3) @ np.array([4, 2, 1])
148     dice = (tri[tri < 6] + 1)[:20]
149
150     if len(vn) < 100:
151         verdict = r"Too few debiased bits for the randomness tests; record a longer run."
152     elif not fails:
153         verdict = (rf"The debiased stream passes all four tests at $\alpha={ALPHA}$: no detectable "
154                  rf"bias or correlation in {fmt(len(vn), 0)} bits.")
155     else:
156         verdict = (rf"The debiased stream fails {len(fails)} of 4 tests at $\alpha={ALPHA}$ "
157                  rf"({'', '.join(fails)}), so it is not a clean random source at this light level.")
158     if not np.isfinite(corr):
159         flicker = "n/a"
160     elif abs(corr) < 0.2:
161         flicker = (rf"The noise on the two photodiodes is nearly uncorrelated ($r={fmt(corr)}$), so it is "
162                  rf"mostly independent per-diode noise (photon shot noise plus read noise) rather than "
163                  rf"flicker of the phone light, which would move both channels together.")
164     else:
165         flicker = (rf"The noise on the two photodiodes is correlated ($r={fmt(corr)}$): part of it is "
166                  rf"flicker of the phone light, which moves both channels together.")
167
168     os.makedirs(FIG, exist_ok=True)
169     plots(t, c0, d0, vn)
170     with open(os.path.join(REPORT, "tests_table.tex"), "w") as f:
171         f.write(tests_table(raw_t, vn_t, len(raw), len(vn)))
172     m = {"isSimulated": int(simulated), "runGain": meta["gain"], "runAtime": meta["atime_ms"],
173         "runSamples": fmt(len(c0), 0), "runMinutes": fmt((t[-1] - t[0]) / 60, 1),

```

```

174         "runMean": fmt(c0.mean(), 0), "runMeanIR": fmt(c1.mean(), 0), "runSigma": fmt(sigma0),
175         "runSigmaIR": fmt(sigma1), "runCorr": fmt(corr), "runRawBits": fmt(len(raw), 0),
176         "runBits": fmt(len(vn), 0), "runBytes": fmt(nbytes, 0),
177         "runYield": fmt(100 * len(vn) / max(len(raw), 1), 0),
178         "runDice": " ".join(map(str, dice)) if len(dice) else "n/a",
179         "runFlicker": flicker, "runVerdict": verdict + (r" \textbf{Warning: the run saturated.}" if sat else "
    ")
180     with open(os.path.join(REPORT, "results.tex"), "w") as f:
181         f.write("% generated by analyze.py - do not edit by hand\n")
182         f.write("".join(f"\\renewcommand{{\\{k}}}}{{{v}}}\n" for k, v in m.items()))
183
184     code_dir = os.path.join(REPORT, "code")
185     os.makedirs(code_dir, exist_ok=True)
186     for fn in ("sensor.py", "collect.py", "analyze.py", "sonify.py", "waves.py", "sonic_pi.rb"):
187         shutil.copy(os.path.join(HERE, fn), code_dir)
188     shutil.make_archive(os.path.join(HERE, "report_overleaf"), "zip", REPORT)
189
190     print(f"\n{'SIMULATED ' if simulated else ''}run: {len(c0)} samples over {(t[-1] - t[0]) / 60:.1f} min")
191     print(f"ch0 mean {c0.mean():.0f} counts, noise {sigma0:.2f} counts ch1 mean {c1.mean():.0f}, noise {
        sigma1:.2f}")
192     print(f"ch0/ch1 noise correlation r = {corr:.2f} ({'mostly per-diode noise' if abs(corr) < 0.2 else '
        light flicker present'})")
193     if sat:
194         print(" WARNING: saturated. Move the phone farther and record again.")
195     if sigma0 < 1.5:
196         print(" WARNING: noise under ~1.5 counts; LSBs are weakly random. Move the phone closer.")
197     print(f"\nRandom numbers: {len(raw)} raw bits -> {len(vn)} clean bits ({nbytes} bytes)")
198     for k, v in vn_t.items():
199         print(f" {k:>8}: p = {v:.3f} {'pass' if v >= ALPHA else 'FAIL'}")
200     print(f" first 64 bits: {' '.join(map(str, vn[:64]))}")
201     print(f" dice rolls : {' '.join(map(str, dice))}")
202     print(f"\nSaved data/random_bits.txt, data/random_bytes.bin; report updated -> report_overleaf.zip")
203     if simulated:
204         print("NOTE: data is SIMULATED; the report will carry a red 'simulated' banner.")
205
206
207 if __name__ == "__main__":
208     main()

```

Listing 4: sonify.py

```

1  """Turn sensor noise into music.
2
3  python3 sonify.py live: sensor -> OSC -> Sonic Pi on this Pi
4  python3 sonify.py --host 192.168.1.20 live, Sonic Pi running on your laptop
5  python3 sonify.py --wav demo.wav offline: render data/run.csv to a WAV file
6  add --simulate to run live without hardware (rehearsal only)
7
8  Mapping: the least-significant bit of each channel is a raw random bit; pairs of
9  raw bits go through a von Neumann extractor (01->0, 10->1, 00/11 dropped) to
10 remove bias. Every 4 clean bits make one note: 3 bits pick one of 8 scale notes,
11 1 bit picks short or long. The light level (ch0) drives the pad's filter, so
12 covering the pinhole audibly darkens the sound.
13 """
14 import argparse
15 import csv
16 import math
17 import os
18 import socket
19 import struct
20 import wave
21
22 import numpy as np
23
24 from sensor import open_sensor, saturation
25
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 SCALE = [57, 60, 62, 64, 67, 69, 72, 74] # A minor pentatonic, MIDI numbers (8 notes = 3 bits)

```

```

28
29
30 class NoteMaker:
31     def __init__(self):
32         self.pending, self.bits = None, []
33
34     def feed(self, ch0, ch1):
35         """Feed one sample; returns a list of (midi_note, duration_s)."""
36         for raw in (ch0 & 1, ch1 & 1):
37             if self.pending is None:
38                 self.pending = raw
39             else:
40                 if raw != self.pending:
41                     self.bits.append(self.pending)
42                     self.pending = None
43         notes = []
44         while len(self.bits) >= 4:
45             b0, b1, b2, b3 = self.bits[:4]
46             del self.bits[:4]
47             notes.append((SCALE[b0 << 2 | b1 << 1 | b2], 1.6 if b3 else 0.5))
48         return notes
49
50
51 def light_level(ch0, sat):
52     """0..1, logarithmic in counts."""
53     return min(max(math.log10(max(ch0, 1)) / math.log10(sat), 0.0), 1.0)
54
55
56 def osc_message(address, *args):
57     """Encode a minimal OSC 1.0 message (int32 / float32 arguments)."""
58     def pad(b):
59         return b + b"\0" * (4 - len(b) % 4)
60     tags, payload = "", b""
61     for a in args:
62         if isinstance(a, int):
63             tags, payload = tags + "i", payload + struct.pack(">i", a)
64         else:
65             tags, payload = tags + "f", payload + struct.pack(">f", float(a))
66     return pad(address.encode()) + pad(tags.encode()) + payload
67
68
69 def live(args):
70     sensor = open_sensor(args.simulate, args.gain, 100, realtime=True)
71     if args.simulate:
72         sensor.set_level(15000)
73     sat = saturation(sensor.ptime_ms)
74     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
75     dest = (args.host, args.port)
76     maker, i = NoteMaker(), 0
77     print(f"Sending OSC to {args.host}:{args.port} (Ctrl+C to stop)")
78     try:
79         while True:
80             _, c0, c1 = sensor.read()
81             lvl = light_level(c0, sat)
82             if i % 5 == 0:
83                 sock.sendto(osc_message("/photon/level", lvl), dest)
84             for note, dur in maker.feed(c0, c1):
85                 sock.sendto(osc_message("/photon/note", note, dur, 0.3 + 0.5 * lvl), dest)
86                 print(f"ch0 {c0:6d} note {note} {'long ' if dur > 1 else 'short'}")
87             i += 1
88     except KeyboardInterrupt:
89         print()
90     finally:
91         sensor.close()
92
93
94 def render_wav(args):
95     path = os.path.join(HERE, "data", "run.csv")

```

```

96     with open(path) as f:
97         meta = dict(kv.split("=", 1) for kv in f.readline().lstrip("#").split())
98         rows = list(csv.DictReader(f))
99     sat = saturation(int(meta["atime_ms"]))
100    maker, events, levels = NoteMaker(), [], []
101    for r in rows:
102        t = float(r["t_s"])
103        if t > args.seconds:
104            break
105        levels.append((t, light_level(int(r["ch0"]), sat)))
106        events += [(t, n, d) for n, d in maker.feed(int(r["ch0"]), int(r["ch1"]))]
107    sr = 22050
108    total = int((min(args.seconds, levels[-1][0]) + 3) * sr)
109    out = np.zeros(total)
110    for t, note, dur in events:
111        f0 = 440 * 2 ** ((note - 69) / 12)
112        n = int((dur + 1.0) * sr)
113        tt = np.arange(n) / sr
114        env = np.minimum(tt / 0.005, 1) * np.exp(-tt / (0.45 * dur))
115        tone = np.sin(2 * np.pi * f0 * tt) + 0.3 * np.sin(4 * np.pi * f0 * tt) + 0.1 * np.sin(6 * np.pi * f0 *
116            tt)
117        s = int(t * sr)
118        seg = out[s:s + n]
119        seg += 0.25 * env[: len(seg)] * tone[: len(seg)]
120    tt = np.arange(total) / sr
121    lvl = np.interp(tt, [x[0] for x in levels], [x[1] for x in levels])
122    out += 0.06 * (0.3 + lvl) * (np.sin(2 * np.pi * 110 * tt) + 0.5 * np.sin(2 * np.pi * 164.8 * tt))
123    out = (out / max(np.abs(out).max(), 1e-9) * 0.9 * 32767).astype("<i2")
124    with wave.open(args.wav, "wb") as w:
125        w.setnchannels(1)
126        w.setsampwidth(2)
127        w.setframerate(sr)
128        w.writeframes(out.tobytes())
129    tag = " (SIMULATED data)" if meta.get("simulated") == "1" else ""
130    print(f"Wrote {args.wav}: {len(events)} notes over {total / sr:.0f} s{tag}")
131
132    def main():
133        p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
134        p.add_argument("--host", default="127.0.0.1", help="machine running Sonic Pi")
135        p.add_argument("--port", type=int, default=4560, help="Sonic Pi OSC input port")
136        p.add_argument("--gain", default="MAX", choices=["LOW", "MED", "HIGH", "MAX"])
137        p.add_argument("--simulate", action="store_true")
138        p.add_argument("--wav", help="render data/run.csv to this WAV file instead of running live")
139        p.add_argument("--seconds", type=float, default=60, help="length of the WAV render")
140        args = p.parse_args()
141        render_wav(args) if args.wav else live(args)
142
143    if __name__ == "__main__":
144        main()
145

```

Listing 5: waves.py

```

1  """Math sound waves from your recorded sensor noise, played on a USB speaker.
2
3  python3 waves.py pluck Karplus-Strong strings, excited by the sensor's own noise
4  python3 waves.py fm FM bells; the noise sets the modulation index (brightness)
5  add --device plughw:1,0 to pick the USB speaker (find it with: aplay -l)
6
7  Reads data/run.csv, writes waves_<mode>.wav, then plays it with aplay.
8  """
9  import argparse
10 import csv
11 import os
12 import subprocess
13 import wave
14

```

```

15 import numpy as np
16
17 from sonify import NoteMaker
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 SR = 22050
21 PHI = (1 + 5 ** 0.5) / 2 # golden ratio: inharmonic, bell-like FM
22
23
24 def load():
25     with open(os.path.join(HERE, "data", "run.csv")) as f:
26         f.readline()
27         rows = list(csv.DictReader(f))
28         t = np.array([float(r["t_s"]) for r in rows])
29         c0 = np.array([int(r["ch0"]) for r in rows])
30         c1 = np.array([int(r["ch1"]) for r in rows])
31         return t, c0, c1
32
33
34 def notes(t, c0, c1, seconds):
35     maker, ev = NoteMaker(), []
36     for i in range(len(t)):
37         if t[i] > seconds:
38             break
39         ev += [(t[i], i, n, d) for n, d in maker.feed(int(c0[i]), int(c1[i]))]
40     return ev
41
42
43 def pluck(f0, dur, excitation, decay=0.996):
44     """Karplus-Strong:  $y[n] = \text{decay} * (y[n-N] + y[n-N-1]) / 2$ , delay line seeded with real noise."""
45     N = max(int(SR / f0), 2)
46     buf = np.resize(excitation, N).astype(float)
47     out, last = [], 0.0
48     for _ in range(int(dur * SR / N) + 1):
49         out.append(buf)
50         buf = decay * 0.5 * (buf + np.concatenate([last, buf[:-1]]))
51         last = out[-1][-1]
52     return np.concatenate(out)[: int(dur * SR)]
53
54
55 def fm(f0, dur, index):
56     """ $y = \sin(2 \pi f t + I(t) \sin(2 \pi \phi f t))$ ; sidebands follow Bessel functions  $J_k(I)$ ."""
57     tt = np.arange(int(dur * SR)) / SR
58     env = np.minimum(tt / 0.005, 1) * np.exp(-tt / (0.35 * dur))
59     return env * np.sin(2 * np.pi * f0 * tt + index * env * np.sin(2 * np.pi * PHI * f0 * tt))
60
61
62 def main():
63     p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
64     p.add_argument("mode", choices=["pluck", "fm"])
65     p.add_argument("--seconds", type=float, default=60)
66     p.add_argument("--device", help="ALSA device for aplay, e.g. plughw:1,0")
67     p.add_argument("--no-play", action="store_true")
68     a = p.parse_args()
69
70     t, c0, c1 = load()
71     dx = np.diff(c0).astype(float)
72     z = (dx - dx.mean()) / (dx.std() + 1e-9) # standardized sensor noise
73     local = np.convolve(np.abs(z), np.ones(25) / 25, mode="same") # rolling noise strength
74     out = np.zeros(int((min(a.seconds, t[-1]) + 4) * SR))
75     for ts, i, n, d in notes(t, c0, c1, a.seconds):
76         f0 = 440 * 2 ** ((n - 69) / 12)
77         if a.mode == "pluck":
78             seg = pluck(f0, d + 1.5, np.resize(z[i:i + 400], 400) if i + 1 < len(z) else z)
79         else:
80             seg = fm(f0, d + 1.0, index=4 * local[min(i, len(local) - 1)])
81         s = int(ts * SR)
82         out[s:s + len(seg)] += 0.3 * seg[: len(out) - s]

```

```

83 out = (out / (np.abs(out).max() + 1e-9) * 0.9 * 32767).astype("<i2")
84 path = os.path.join(HERE, f"waves_{a.mode}.wav")
85 with wave.open(path, "wb") as w:
86     w.setnchannels(1), w.setsampwidth(2), w.setframerate(SR)
87     w.writeframes(out.tobytes())
88 print(f"Wrote {os.path.basename(path)} ({len(out) / SR:.0f} s)")
89 if not a.no_play:
90     subprocess.run(["aplay"] + (["-D", a.device] if a.device else []) + [path])
91
92
93 if __name__ == "__main__":
94     main()

```

Listing 6: sonic_pi.rb

```

1 # Photon-noise sonifier: paste into a Sonic Pi buffer, press Run, then start sonify.py.
2 # Notes arrive as OSC /photon/note [midi, duration, amp]; the light level as /photon/level [0..1].
3
4 set :cut, 90
5
6 live_loop :notes do
7     use_real_time
8     n, dur, amp = sync "/osc*/photon/note"
9     with_fx :reverb, room: 0.8, mix: 0.4 do
10         synth :pretty_bell, note: n, release: dur, amp: amp
11     end
12 end
13
14 live_loop :level do
15     use_real_time
16     lvl = sync "/osc*/photon/level"
17     set :cut, 55 + 75 * lvl[0] # brighter light -> brighter pad
18 end
19
20 live_loop :pad do
21     synth :hollow, note: :a2, attack: 2, sustain: 2, release: 4, cutoff: get(:cut), amp: 0.7
22     sleep 4
23 end

```